

김소현 Kim Sohyeon

약력

쿠팡 Backend Engineer (2024.11 ~)

학력

중앙대학교 (2019.03 ~ 2023.08)

전공: 소프트웨어학부

Work

쿠팡 Coupang (2024.11 ~ 현재)

Core Serving Platform / Backend Engineer

Tech

Java · Spring Boot · Spring WebFlux

Core 및 A/B Test Feature 개발

상품·가격·혜택·배송 등 E-commerce의 다양한 도메인 개념을 이해하고, 여러 Feature Team의 요구사항을 분석하여, Core 데이터 기반의 비즈니스 로직을 빠르고 안정적으로 구현했습니다.

- 실험(A/B Test) 로직을 기존 Core 로직과 분리하고 비동기로 실행되도록 구성하여, Core Logic 흐름에 미치는 영향을 최소화했습니다.
- 기존 Core Logic의 데이터 흐름과 I/O 구간을 분석해 실험 로직의 최적의 실행 시점(Trigger Point)을 결정하고, 불필요한 응답 지연이 발생하지 않도록 구성했습니다.
- 실험이 정식 Feature로 전환되면 기존 Core Logic에 미치는 영향을 최소화하면서도 특정 Feature에 종속되지 않고 확장할 수 있는 구조로 반영했습니다.

Feature1. 무이자 할부 정보 제공

사용자의 카드 정보와 상품 가격을 기반으로 적용 가능한 무이자 할부 개월 수와 월별 결제 금액을 제공하는 A/B 비즈니스 로직을 구현했습니다.

- 변경 빈도가 낮은 카드사별 할부 정보를 Local Cache에 저장하고 10분 주기로 갱신하여, 요청마다 할부 데이터를 조회하지 않도록 구성했습니다.
- 사용자 카드 코드와 상품 가격을 기준으로 적용 가능한 할부 조건을 판단하고 월별 결제 금액을 계산하는 로직을 구현했습니다.

Feature2. 경제적인 상품 옵션 추천

검색 결과에서 동일 상품의 더 경제적인 수량 옵션을 안내하는 A/B 비즈니스 로직을 구현했습니다.

- 현재 옵션보다 수량이 많은 옵션을 대상으로 개당 가격을 비교해 가장 경제적인 옵션과 절감 금액을 계산했습니다.
- 반올림·올림·내림 기준, 동일 단가 옵션 처리, 1개 구성 옵션 노출 여부 등 계산·노출의 경계 조건을 케이스별로 정리하고 관련 팀과 기준을 조율했습니다.
- 실험 로직 구현체의 generate 메서드에 계산 로직을 구현하고, 데이터 의존관계를 기준으로 실행 시점과 Core 응답에 결과를 병합하는 시점을 결정했습니다.

기존 도메인을 유지하며 쿠폰·혜택 정책 반영

공통 할인·혜택 로직의 변경은 대상 정책 외 기존 동작에도 영향을 줄 수 있어, **기존 도메인 의미를 유지**하면서 정책이 달라지는 지점만 분리해 반영했습니다.

Feature1. 체험 프로모션 쿠폰의 할인 순서 분리

특정 체험 프로모션의 다운로드 쿠폰은 일반 다운로드 쿠폰과 동일한 유형이지만, 정책상 즉시할인보다 먼저 적용되어야 했습니다. (*일반 다운로드 쿠폰은 즉시할인 적용 금액에 반영)

- 모든 100% 쿠폰의 우선순위를 변경하거나 새로운 프로모션 Type을 추가하는 방안을 함께 검토했습니다.
- 전체 우선순위 변경은 무관한 쿠폰까지 영향을 주고, 새로운 Type을 추가하기에는 우선순위 외의 모든 다운로드 쿠폰 정책과 동일해 과도하다고 판단했습니다.
- 기존 프로모션 Type은 유지하고 대상 체험용 정책만 식별해 **전용 할인 계산 구현체를 선택하도록 분리**했습니다.

Feature2. 멤버십 혜택의 발급 판단 수정

상호배타적인 가입 혜택 중 하나를 사용했음에도 다른 가입 혜택이 발급 가능하거나 배너에 노출되는 문제가 있었습니다.

- 사용 가능 여부와 발급 가능 여부가 서로 다른 조건으로 판단되는 로직을 확인했습니다.
- 새로운 프로모션 Type을 추가하지 않고 기존 상호배타 Group과 Counter를 재사용해 **상호배타적인 사용 이력을 발급 판단에도 반영하여 발급 가능 횟수를 재계산**했습니다.
- 정책 조합을 검토하는 과정에서, 가입 혜택 쿠폰을 미리 발급받고 미사용한 경우 배너가 계속 노출될 수 있는 **Edge Case**를 발견했습니다. 이를 PO에게 공유하고 배너 노출 담당 팀에 조건 추가 필요를 인지할 수 있게 했습니다.

AI Agent를 활용한 반복 운영 업무 자동화

단발성 A/B Test 실험 로직을 백오피스 툴을 통해 운영하면서 설정 확인, 기간 연장 검토, 종료 후 정리 같은 반복 업무가 계속 발생했습니다. 요청마다 조건과 판단 기준이 달라 단순 스크립트로는 일괄 처리하기 어려웠습니다.

[문제 상황]

- 엔지니어가 요청별로 실험 상태와 조건을 확인하고, 연장 필요 여부와 종료 후 정리 대상을 직접 검토했습니다.
- 초기에 AI Agent를 활용하여 반복 업무 자동화를 시도했으나, 운영 시스템 접근 시 기존 인증 권한 체계를 그대로 유지해야 했습니다.

[해결 과정]

- 사내 AI Agent 플랫폼을 활용해 **업무용 Agent**를 구성하고, **백오피스 연동용 MCP Server**를 직접 개발했습니다.
- MCP 전용 인증정보와 기존 백오피스 사용자 권한을 함께 검증하도록 구현했습니다.
- Agent 실행 시, 승인 및 실험 로직 배포 권한은 whitelist 기준으로 제한하여, **기존 권한 정책은 유지**했습니다.
- Agent는 실험 정보 조회와 조건 분석을 보조하고, 타당성 검증 결과에 따라 리뷰 결과와 함께 승인자/요청자를 호출하도록 했습니다. 엔지니어는 1차 검증된 요청만을 검토함으로써 요청자와의 **커뮤니케이션 비용을 줄였습니다**.
- Agent 구성 시, Security Review를 진행하고, 관련 절차를 문서화하여 다른 팀원들도 Agent 구성에 참고할 수 있도록 했습니다.

Production 장애 분석 및 Migration Regression 해결

실서비스 이슈가 발생하면 화면의 결과만 보고 원인을 추측하기보다, 로그·지표·Trace와 실제 데이터 흐름을 따라 정상 구간을 하나씩 확인하며 문제 범위를 좁혔습니다.

Case1. eGift 주문 Migration Regression

과거 Migration 이후, eGift 주문에서 수령인의 우편번호가 배송예정일(PDD) 계산에 반영되지 않는 문제가 일부 요청에서 발생했습니다.

- Request Log에서 수령인 ZipCode가 정상적으로 들어오는지 먼저 확인해 입력 단계는 정상임을 확인했습니다.
- 이후 데이터 변환·계산 흐름을 따라가며 **정상 동작하던 Legacy 코드와 Migration 이후 코드를 비교해 누락된 조건을 발견**했습니다.
- 원인 파악 후 팀 내 상황을 공유하여 히스토리부터 파악했습니다.
- 과거 Migration 중 누락된 사실 확인 후에, 누락된 조건을 Migration 후 로직에 추가했습니다. 해당 시나리오를 기반으로 테스트 코드를 추가하여 문제 해결을 확인하고, hot-fix 했습니다.

Case2. Availability Drop Incident 분석

장애 대응 중 특정 API Availability가 감소하고 499가 증가했지만, 초기 Metric에서는 특정 Upstream 서비스의 Latency 증가가 명확하게 보이지 않았습니다.

- 서비스 Metric과 장애 당시 Timeline을 확인하고, 후보 시스템을 하나씩 검증했습니다.
 - Trace를 따라 요청 흐름을 확인해 Kafka Produce 구간까지 원인 후보를 좁혔습니다.
 - 실제 원인 코드 및 개선은 관련 팀의 범위였음을 확인하고, 영향 범위 파악과 원인 후보 분석, Incident Report 및 관련 팀 커뮤니케이션까지 담당했습니다.
-

Project

피키 (2024.04 ~ 2024.10)

모임 장소 추천 서비스 / Backend Developer / [Github](#)

Tech

Kotlin · Spring Boot 3.3 · JPA · PostgreSQL · Redis · Docker · GitHub Actions · NCP

Team

Designer 2 · Frontend 3 · Backend 3

외부 Distance API 반복 호출을 Cache로 개선

[문제 상황]

- 피키는 특정 장소 간의 거리를 포함한 기능을 제공합니다. 각 장소의 위치 정보(경도, 위도)를 기반으로 OpenAPI를 호출하여 거리 정보를 조회합니다.
- 또한, 하루에 OpenAPI 호출 제한이 있었습니다. 따라서 조회 성능을 개선하고, API 호출을 최소화 할 수 있는 캐싱 대책이 필요했습니다.

[해결 과정]

- 2가지 해결방안 중 **a 방안**을 선택했습니다. **캐싱 전략**이기 때문에 데이터베이스에 주는 부하를 최소화할 수 있고, b 방안보다 더 빠른 성능을 낼 수 있다는 장점을 높게 평가했습니다. 또한, 최신 데이터를 100% 보장하지 못한다는 단점이 있지만, 자주 변동되지 않는 데이터 특징과 캐시 만료 시간 설정으로 충분히 보완 가능한 점이라고 생각했습니다.
 - 캐시를 활용하여 특정 장소 간의 거리 데이터를 캐싱합니다. ([Tech Blog](#))
 - 데이터베이스 칼럼 또는 테이블을 추가하여 특정 장소 간의 거리 데이터를 저장합니다.
- 캐싱된 데이터에 다수의 사용자가 접근할 수 있기 때문에 **더 정확한 일관성을 보장하는 Redis**에서 제공하는 캐시를 활용했습니다.
 - 자원 비용을 절약하기 위해 일부 서비스를 무료로 제공하는 upstash에서 Redis 데이터베이스를 생성하여 활용했습니다.
- @Cacheable 어노테이션이 적용되지 않는 문제**를 마주했습니다.
 - 스프링에서는 프록시 패턴을 통해 캐시 로직이 적용되는데, 동일 클래스 내에서 호출되는 메서드는 프록시가 적용되지 않는 구조가 원인임을 파악했습니다.
 - 프로젝트 구조는 비즈니스 로직을 담당하는 도메인 클래스와, 외부 API 호출을 담당하는 어댑터 클래스가 분리되어 있습니다. 따라서 **어댑터 클래스 내부의 메서드에 어노테이션을 추가**하여 캐싱 로직이 정상적으로 동작할 수 있도록 했습니다.
- 성능 테스트를 통해 만료된 캐시 데이터에 다종의 요청 접근으로 인해 **Cache Stampede** 현상으로 인해 과부하가 발생하는 문제를 마주했습니다.
 - Redisson 분산락**을 활용하여 문제를 해결했습니다. 캐시 miss가 발생했을 경우 첫번째 요청이 락을 획득하여 캐시를 갱신하고, 나머지 요청은 대기 후 캐싱된 데이터를 조회할 수 있도록 했습니다.

[결과]

- 동일 거리 정보에 대한 반복 외부 호출을 줄이는 구조로 최적화했습니다.
- 당시 Redis를 사용했지만, 현재 다시 판단한다면 실제 단일 서버 구성과 캐시 공유 필요성을 보고 Local Cache도 함께 고려할 수 있습니다.

Project

스밈 (2023.05 ~ 2024.10)

영어 일기 서비스 / Backend Developer / [Github](#)

Tech

Java17 · Spring Boot 3.2.1 · JPA · PostgreSQL · Redis · Docker · GitHub Actions · AWS EC2

Team

Plan · Designer · Mobile Developer(Android/iOS) · Backend

비트맵 캐시를 활용한 방문 수 데이터 최적화

[문제 상황]

스밈에서는 사용자의 동기부여를 위해 누적 방문 수 정보를 제공합니다. 이러한 기능을 구현하기 위해 방문 내역 테이블을 만들어서 데이터를 저장했습니다. 하지만 하루 최대 600개 이상의 데이터가 쌓여갔고, 데이터의 양을 최적화 할 방법이 필요했습니다.

[해결 과정]

- 먼저 기획자와 소통하여 방문 내역 데이터의 필요성에 대해 논의했습니다. 이미 GA(Google Analytics) 기능과 일기 작성 기능을 통해 트래킹 조치가 가능하기 때문에, **방문 내역 데이터까지는 필요없다는 결론**을 내렸습니다. 따라서 방문 횟수 데이터만 최적으로 저장할 방법을 고민했습니다.
- 회원 테이블에서 방문 횟수 칼럼으로 해당 기능을 구현할 수 있지만, **금일 방문한 내역이 있으면 칼럼을 업데이트하지 않는 검증 로직**이 필요했습니다. 따라서 데이터를 최적으로 관리할 수 있으면서, 금일 방문 여부를 조회할 수 있는 방안 2가지를 떠올렸습니다.
- b 방안**을 선택했습니다. a 방안은 Batch를 실행하는 시점에 방문 업데이트 요청을 처리하게 되면 데이터의 정합성을 보장하지 못할 것이라 생각했습니다. 하지만 b 방안의 비트맵 데이터를 매일 생성하여 관리하면 **정합성 문제를 해결**할 수 있습니다. 또한, 비트맵 데이터는 데이터를 비트 단위로 저장하기 때문에, **메모리 효율성과 빠른 데이터 연산**이 가능한 장점이 있습니다.
 - a. 회원 테이블에 금일 방문 여부 칼럼을 추가하고, 매일 자정에 Batch를 돌려 초기화합니다.
 - b. 비트맵(Bitmap) 자료구조를 활용하여, 금일 방문 여부를 조회할 수 있도록 합니다.
- 데이터베이스의 부하를 최소화하고, 금일 방문 내역 조회 성능을 개선하기 위해 **비트맵 데이터를 캐싱**했습니다.
 - 날짜를 기반으로 key를 생성했습니다. (key: visit:yyyyMMdd , value: 비트맵 데이터)
 - 캐시 데이터베이스**로 Redis를 선택했습니다. Redis에서 기본으로 비트맵 자료구조를 제공하며, 모든 회원의 방문 데이터를 포함하기 때문에 **일관성 보장**이 중요했습니다.
 - 키 만료 시간을 2일로 설정하여, 하루동안 정확한 방문 여부 조회가 가능하게 했습니다.

[결과]

- 방문 이벤트를 Row로 계속 저장하던 구조를 회원별 Bit 상태 표현으로 최적화했습니다.
- 당시에는 Redis의 당일 유실이나 DB 반영 실패에 대한 복구 전략까지 고려하지 못했습니다. 현재 다시 설계한다면 집계 작업의 재실행 가능성과 Redis 유실 시 복구 기준도 함께 검토할 수 있습니다.

Project

Docker · Nginx 기반 Blue-Green 배포 구조 구현

[문제 상황]

- 초기에는 GitHub Actions + AWS CodeDeploy를 사용해 JAR 파일과 실행 Script 중심으로 배포했습니다.
- 새 버전을 배포할 때 기존 프로세스를 교체해야 해, 서비스 운영 중 발생하는 다운타임을 줄이고 싶었습니다.

[해결 과정]

- Github Actions를 활용하여 배포를 자동화할 수 있는 **CI/CD 파이프라인을 구축**했습니다. ([Tech Blog](#))
 - **자원을 절약하기 위해** t2.micro 인스턴스를 사용했습니다. 배포 자동화에 필요한 요소는 모두 갖추고 있으며, GitHub의 클라우드 자원을 사용하므로 적합한 선택이라고 판단했습니다.
 - 배포 스크립트를 작성하여 github actions 클라우드에서 새로운 코드가 배포되도록 설정했습니다. 배포가 자동화되어 안정적인 배포가 가능해졌습니다.
- Gradle의 dependencies에 종속성이 추가될수록 jar 파일의 크기가 커져, 빌드 및 배포 시간 증가했습니다.
 - Docker image를 활용하여 배포 과정을 최적화했습니다. 레이어 재사용과 멀티스테이지 빌드를 통해 최적화된 이미지를 생성하고, 경량화된 베이스 이미지로 배포 시간을 단축했습니다.
 - CD 배포 스크립트를 작성하여 스프링 빌드 파일을 docker image로 빌드했습니다.
 - 빌드된 docker image를 docker hub에 push했습니다.
 - 배포 명령어를 통해 EC2에서 docker image를 pull하고, 컨테이너를 실행하여 서비스가 배포되도록 했습니다.
- 이후 배포 단계에서 잠시 서버가 다운되는 현상이 발생했습니다. 서비스의 가용성을 높이기 위해 **블루그린 배포 전략**을 활용하여 **무중단 배포**가 가능하게 CI/CD 파이프라인을 수정했습니다.

[결과]

- 배포 단위를 Docker Image로 통일하여 유지보수성을 높였습니다.
 - 신규 버전의 정상 동작을 확인한 후 트래픽을 전환하는 구조를 직접 구현했습니다.
-

Project

소프티 (2024.01 ~ 2024.09)

루틴 관리 서비스 / Backend Developer / [Github](#)

Tech

Java17 · Spring Boot 3.2.1 · JPA · PostgreSQL · Redis · Docker · GitHub Actions · AWS EC2

Team

Plan · Designer(2) · Mobile Developer(Android/iOS) · Backend(2)

삭제·복원 요구사항을 고려한 Soft Delete 구조 재설계

[문제 상황]

- 사용자가 루틴을 삭제한 뒤 같은 루틴을 다시 추가하더라도, 기존의 누적 달성 기록을 복원해야 했습니다.
- Hard Delete를 적용하면 데이터가 사라지기 때문에, 삭제된 데이터의 상태를 유지할 수 있는 구조가 필요했습니다.

[1차 해결 과정]

처음에는 일반 조회마다 isDeleted 조건이 추가되는 것을 피하기 위해 현재 루틴과 삭제된 루틴을 **별도 Table로 분리**했습니다.

- 삭제 시 기존 달성 기록을 삭제 전용 Table로 이동
- 재추가 시 삭제 Table에서 기존 기록 조회
- achievementCount 등의 상태를 복원한 뒤 현재 루틴 데이터 재생성

[한계점 발견]

실제 구현·유지보수 과정에서 Table 분리로 인한 복잡도가 더 커졌습니다.

- 삭제·복원마다 두 Table 사이에서 데이터 이동
- 데이터 생성·삭제 로직 증가
- 현재/삭제 상태를 확인하려면 두 Table을 함께 확인해야 함
- Edge Case가 늘어날수록 처리 흐름이 복잡해짐

→ 위와 같이, 한 테이블에서 isDeleted 조건을 관리하는 비용보다 Table 이중화와 데이터 이동 비용이 더 크다고 판단했습니다.

[2차 해결 과정]

두 Table을 하나로 통합하고 **is_deleted 기반 Soft Delete**로 재설계했습니다.

- @SQLDelete: DELETE 요청을 실제 삭제 대신 is_deleted = true로 변경
- @SQLRestriction: 일반 조회에서는 삭제된 데이터를 자동 제외
- 삭제된 데이터를 복원해야 하는 경우
 - Native Query로 삭제 데이터를 명시적으로 조회
 - 기존 데이터를 다시 활성화해 과거 achievementCount 등 상태 유지

[결과]

- 삭제·복원 과정에서 발생하던 Table 간 데이터 이동을 제거했습니다.
 - 현재 상태와 삭제 상태를 하나의 Entity/Table에서 관리하고, 기존 달성 기록을 유지하면서 삭제·복원 로직을 단순화했습니다.
 - 처음 선택한 구조를 유지하기보다, 실제 구현 후 드러난 관리 비용을 기준으로 더 단순한 데이터 모델로 재설계했습니다.
-

Experience

IT 연합 동아리 Mash-Up

2024.03 ~ 2024.10 / Spring Platform / <https://mash-up.kr>

백엔드 개발(Spring Platform) 담당으로 '피키' 서비스를 개발했습니다. 기능 구현에 그치지 않고, 사용자의 참여 흐름과 서비스의 목적을 함께 고려하며 기능을 설계했습니다.

- 피키: 투표 기반으로 모임 장소 코스를 정하는 웹앱 서비스
- 유저들이 **투표에 적극적으로 참여할 수 있도록 참여 과정을 단순화하는 방법**을 고민했습니다.
 - 단독방 링크 공유나 푸시 알림 등 리마인드 방식도 검토했지만, 사용자가 모임 장소를 직접 검색하고 판단해야 하는 과정 자체가 부담이라고 판단했습니다.
 - 별도의 검색 없이 판단할 수 있도록 장소의 대표 사진(메뉴)과 기본 정보를 카드 형태로 제공하는 방식을 제안했습니다.
 - 최종적으로 카드에서 O/X를 선택하는 방식으로 구현해 **투표에 필요한 탐색과 입력 과정을 줄였습니다**.

대학생 연합 IT 벤처 창업 동아리 SOPT

2022.03 ~ 2024.01 / Server Part / <https://www.sopt.org>

백엔드 개발(Server Part) 담당으로 '스밈'과 '동아리 출석 서비스'를 개발했습니다. 요구사항을 백엔드 관점에서 검토하고, 서비스 운영과 확장성을 고려해 데이터 구조와 기능을 설계했습니다. 또한 기획자·디자이너·클라이언트 개발자와 요구사항을 조율하며 서비스를 출시했고, 이후 운영·유지보수까지 경험했습니다.

- 스밈: 영어 훈련을 위한 영어 일기 앱 서비스
 - 서비스를 약 1년간 운영하며, 사용자가 영어 일기를 작성하는 것에서 더 나아가 **일상에서 접한 표현을 학습에 활용**할 수 있는 기능을 고민했습니다.
 - 짧은 영상 콘텐츠에서 접한 **영어 표현을 추출·저장하고 일기 작성에 활용**할 수 있는 기능을 구현했습니다.
- SOPT 출석 서비스: 동아리 출석 관리 서비스
 - 기수별 약 200명이 사용하는 출석 서비스를 초기 개발하고 약 1년간 유지보수했습니다.
 - 운영진이 관리자 페이지에서 **출석 내역을 관리**하고, 동아리원이 앱에서 자신의 **출석 현황을 확인**할 수 있도록 구현했습니다.
 - 한 세미나에서 여러 번 출석 체크할 수 있는 요구사항을 고려해 회차(round) 단위로 출석을 기록할 수 있도록 데이터 구조를 설계했습니다.

구름톤 in JEJU

2023.03.28 ~ 03.31 / 57i Backend / <https://9oormthon.goorm.io>

백엔드 개발자로 '탐라주' 서비스를 개발했습니다. 짧은 해커톤 일정 안에서 핵심 사용자 경험을 중심으로 **MVP 범위를 조율하고 서버 기능을 구현**했습니다.

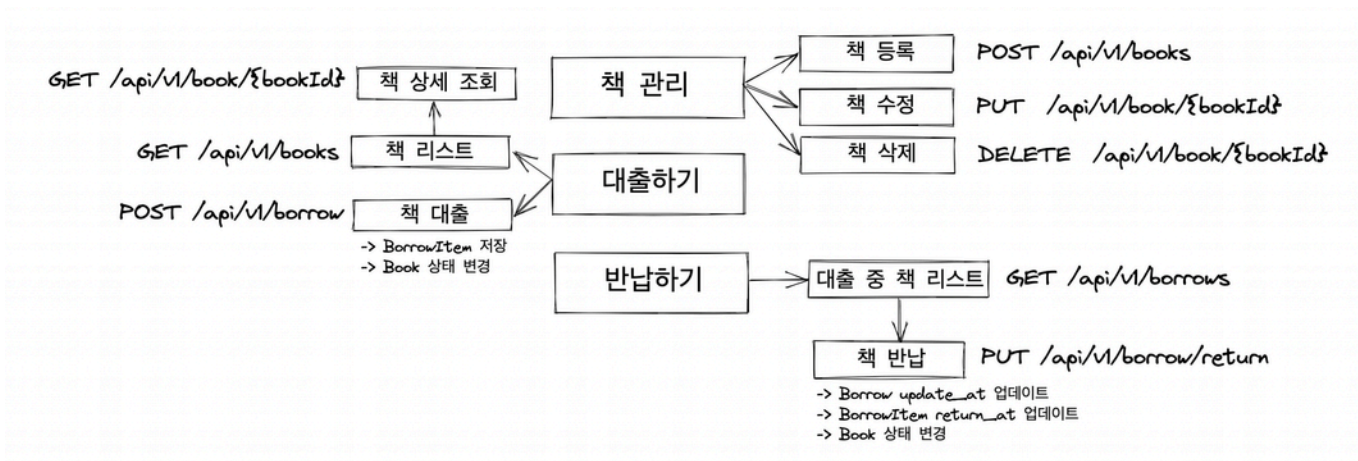
- 탐라주: 제주 소규모 양조장의 전통주 정보와 테이스팅 노트를 제공하고, 마셔본 술을 기록할 수 있는 아카이빙 서비스
 - **제한된 개발 시간**을 고려해 핵심 기능과 데이터 전달 흐름을 우선순위로 정해 MVP를 구현했습니다.
 - 사용자가 전통주를 아카이빙하고 좋아요를 남길 수 있도록 구현했으며, 좋아요 수를 기반으로 인기 전통주를 확인할 수 있는 랭킹 기능을 개발했습니다.
 - 기획자·디자이너·프론트엔드 개발자와 기능 범위를 조율하며 짧은 기간 내 시연 가능한 서비스를 완성했습니다.
 - **구름 기업상(우수상)**을 수상했습니다.
-

Experience

프로그래머스 데브코스 백엔드 코스

2022.10 ~ 2023.03 / Backend / Java, Spring

- Java와 Spring Boot를 활용하여 **추천 음악 대결 서비스**를 주제로 팀 과제를 수행했습니다.
 - Redisson 분산 락을 적용해 좋아요 기능의 **동시성 문제를 해결**했습니다.
 - 동시 요청을 순차적으로 처리하도록 제어해 좋아요 수의 **정합성을 보장**했습니다.
- Spring Boot와 React를 활용하여 **도서 대여·반납 시스템**을 주제로 개인 과제를 수행했습니다.
 - Spring Boot 기반으로 도서 등록, 조회, 수정, 삭제, 대여, 반납 **API를 구현**했습니다.
 - React를 활용하여 도서 목록, 상세 조회, 대여 현황을 확인할 수 있는 **UI를 구현**했습니다.



*도서 대여·반납 시스템의 주요 사용자 흐름과 API 설계

: 책 관리, 대출, 반납 흐름을 API 단위로 정리하며 기능 간 책임과 상태 변화를 설계했습니다.